Field Hours

CSCE 666 Pattern Analysis

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Feature vectors

You turn a measurement into a vector before a classifier sees it, and you choose the coordinates. A 28 by 28 grayscale digit is 784 intensities. Two images of the same digit can sit far apart in that space after a shift of only 2 pixels, so raw distance is not identity. You either build features that absorb the shift, or you leave that work to the later model.

Standardization is a concrete feature transform you fit on training data only. Subtract each coordinate's training mean and divide by that coordinate's training standard deviation, so a coordinate that spanned about 1000 units and one that spanned 0.1 both end with variance 1 on the training split. Using 50 test points to set the mean leaks the evaluation into the representation. You freeze the scale before you score those points.

From a measurement to a model input.

Record the raw signal -> Choose the coordinates -> Fit scale on training data -> Pass the vector onward

Two coordinates, population scale

1. Coordinate A has training values 0 and 2. The mean is 1 and the population standard deviation is 1. Coordinate B has 0 and 4. The mean is 2 and the population standard deviation is 2.

2. A new raw point (2, 4) centers to (1, 2).

3. Dividing by those scales yields (1, 1). You computed both means and both scales without the new point.

Problem 1. Using the training scales in the example, what is the scaled vector of the raw point (2, 4)?

Answer. (1, 1). Centered it is (1, 2), then each coordinate is divided by its training standard deviation, 1 and 2.

Problem 2. You recompute the mean using the training set plus 50 test points, then report test error. What leaked?

Answer. The test measurements entered the feature transform through the mean. Test error no longer scores a representation that was fixed before those points were seen.

Lecture 2 · 90 minutes

Density estimates

A probability report needs a density, or at least a conditional density you estimated. A histogram with equal bins

estimates height by the count in a bin divided by the bin width times the number of samples. With 100 samples and 20 of them in a bin of width 0.1, the height is 20 divided by 10, so 2. An empty bin gets height 0, which breaks a later logarithm.

A kernel estimate places a bump of bandwidth h on every sample and averages them. With a box of width h, a location covered by k of n bumps has density k divided by n times h. You pick h by held-out likelihood, not by how smooth the picture looks. On data that varies on a scale of about 1, a bandwidth of 0.01 memorizes the sample, and a bandwidth of 10 flattens it.

Two density estimators.

Histogram

Fixed bins

Count over width times n

Empty bins are zero

Box kernel

A bump on each sample

Bandwidth h sets the width

Overlap adds the bumps

Four samples, box width 1

1. Samples sit at 0, 0, 1, and 1, so n is 4. Each box kernel has height 1 on an interval of length 1 starting at the sample: [0, 1) or [1, 2).

2. At 0.5 only the two samples at 0 cover the query. Density is 2 divided by (4 times 1), so 0.5.

3. At 2.5 no kernel covers the query, so the density is 0. A log likelihood there is undefined. A wider box, or a Gaussian bump, would have left a positive number.

Problem 1. In the histogram, 100 samples put 20 points in a bin of width 0.1. What height do you plot?

Answer. 2. The count 20 is divided by 100 times 0.1.

Problem 2. In the box-kernel example, what is the density at 0.5, and why is density 0 at 2.5 a modeling choice?

Answer. 0.5 at the point 0.5, because two of four kernels cover it and h is 1. Density 0 at 2.5 only means no kernel reaches there. It is not evidence that future points cannot land there. A wider bandwidth would assign them a small positive density.

Lecture 3 · 90 minutes

Bayes and linear decisions

For 0-1 loss you pick the class with the larger posterior. With equal priors of 0.5, that is the class with the larger class-conditional density. If class A is Gaussian with mean 0 and variance 1, and class B is Gaussian with mean 2

and variance 1, the two densities match at 1, halfway between the means. You place the boundary there because the shared variance cancels.

When you do not trust that Gaussian story, a linear score still cuts the space with a hyperplane. Take weights (1, 0) and bias -1. You label a point positive when the first coordinate minus 1 is positive, so the boundary is the line where the first coordinate equals 1. The second coordinate never moves the score. Training accuracy of 100 percent on 8 points does not tell you the error on a fresh set of 20.

A linear score.

1. Dot the weights with x
2. Add the bias
3. Read the sign
4. Score a fresh split

Two points, one unused coordinate

1. Weights are (1, 0) and the bias is -1. The score is the first coordinate minus 1.

2. The point (1.5, 4) scores 0.5, so you assign the positive class. The point (0.5, 4) scores -0.5, so you assign the negative class.

3. Both points share the second coordinate 4, and it changes nothing because its weight is 0. A scatter plot can make that coordinate look informative while the score ignores it.

Problem 1. What is the score of (1.5, 4), and which class do you assign if a positive score means class 1?

Answer. The score is 0.5, so class 1.

Problem 2. Priors are equal. The two unit-variance Gaussians have means 0 and 2. A point arrives at 1. Which class does the Bayes rule prefer?

Answer. Neither. The densities are equal at 1, so both posteriors are 0.5. You need a tie break. The boundary sits there because the means differ by 2 and the variances match.

Lecture 4 · 90 minutes

Clustering

K-means assigns each point to the nearest of k centers, then replaces each center by the mean of its points. You repeat until assignments stop changing. With k = 2 and points at 0, 0, 10, and 10, centers settle at 0 and 10 if you start one center in each pair. A bad start can park both centers in one clump. You run 10 random initializations and keep the lowest total squared distance.

A Gaussian mixture softens the assignment. Each point carries a responsibility between 0 and 1 for each component, and the mean update weights the point by that number. A responsibility of 0.75 sends three quarters of the point's mass to component 1 and 0.25 to component 2. K-means is the hard case, where every responsibility is 0 or 1. You choose k by a criterion you wrote down before looking, because the training cost falls as k grows.

One k-means pass.

1. Assign points to nearest centers

2. Replace each center by a mean

3. Stop when labels hold

Three points, two centers

1. Points are 0, 2, and 10. You start centers at 0 and 9.

2. Distances put 0 and 2 on the first center and 10 on the second. The new centers are the mean of 0 and 2, which is 1, and the point 10.

3. The next assignment does not move. Squared distances are 1, 1, and 0, so the total is 2. That is the objective you compare across initializations, not a count of clusters you liked in a plot.

Problem 1. What total squared distance do you get at convergence in the example?

Answer. 2. The errors are 0 minus 1, 2 minus 1, and 10 minus 10.

Problem 2. You set k = 3 on these 3 points and reach squared distance 0. Why is that not evidence of better structure?

Answer. Three centers can sit on the three points and drive the training objective to 0 by memorizing them. The objective falls as k grows. You needed a held-out criterion or a penalty on k.

Lecture 5 · 90 minutes

Spectral structure

Principal component analysis finds orthogonal directions of largest variance after you center the data. You form the covariance and take the leading eigenvectors. If the top component holds 0.9 of the trace, the reconstruction you just accepted throws away 0.1. You discard a coordinate only after you read that fraction. A plot of the first two scores is not by itself a reason to drop the rest.

Spectral clustering builds a similarity graph, forms a Laplacian, and clusters eigenvectors instead of the raw coordinates. Two moons that interleave are not two round blobs, and k-means in the raw plane cuts them with a straight boundary between centers. With 2 well separated components, the leading Laplacian eigenvectors are nearly constant on each component. You then run k-means with k = 2 in that eigenvector coordinate.

Variance axes versus a graph cut.

Principal components

Center the coordinates

Take top eigenvectors

Read the variance fraction

Spectral clustering

Build a similarity graph

Take Laplacian eigenvectors

Cluster those coordinates

Four corners of a rectangle

1. Points are (0, 0), (2, 0), (0, 1), and (2, 1). The mean is (1, 0.5). Centered x coordinates are -1 or 1, so their mean square is 1. Centered y coordinates are -0.5 or 0.5, so their mean square is 0.25. Cross terms average to 0.

2. The first axis is horizontal. Its share of the trace is 1 divided by 1.25, so 0.8.

3. Projecting onto x keeps 0.8 of the variance and drops 0.2. You can quote that split. You should not say the vertical coordinate was noise unless a later task agrees.

Problem 1. What fraction of variance does the first component keep in the rectangle example?

Answer. 0.8. The x mean square is 1, the y mean square is 0.25, and 1 divided by 1.25 is 0.8.

Problem 2. Why can k-means on raw coordinates fail on two interlocking moons when spectral clustering with k = 2 succeeds?

Answer. K-means assigns each point to the nearest center, so the boundary between two centers is a straight cut in the raw plane. Interlocking moons are not separated by that cut. Spectral clustering first maps points into Laplacian eigenvector coordinates, where the two components pull apart, and only then runs k-means.

Lecture 6 · 90 minutes

Hidden Markov models

A hidden Markov model has a state you do not see and an observation that depends only on the current state. Given that state, earlier states say nothing about the next one. With 3 hidden states the transition matrix is 3 by 3 and each row sums to 1. Filtering asks for the current state given observations so far. Smoothing uses the whole sequence, including later observations. You do not report a filter number as a smoothed label.

You run the forward step by multiplying the predicted state masses by the observation likelihood, then you normalize. Start from a prior of 0.5 on H and 0.5 on L, before any transition. A symbol with likelihood 0.8 in H and 0.2 in L gives unnormalized masses 0.4 and 0.1. Their sum is 0.5, so the posterior is 0.8 on H and 0.2 on L. Drop the division and the next step is no longer a probability.

One forward filter step.

Predict with the transition -> Multiply by the likelihood -> Normalize the masses -> Emit the current posterior

One observation, then a transition

1. After the observation above, the posterior is 0.8 on H and 0.2 on L.

2. Stay probability is 0.75 and switch probability is 0.25. The next predicted mass on H is 0.8 times 0.75 plus 0.2 times 0.25, so 0.65. The mass on L is 0.35.

3. Those two numbers sum to 1, so this predict step does not need a new normalize. The next observation will. Smoothing this same time index would also fold in any symbols that arrive after it, which this forward number has not seen.

Problem 1. After the first observation and before any transition, what is the posterior on H?

Answer. 0.8. The unnormalized masses are 0.4 and 0.1, and 0.4 divided by 0.5 is 0.8.

Problem 2. Why is smoothing not the same query as the forward pass you just ran?

Answer. The forward pass uses only observations up to the current time. Smoothing a past state also uses observations that came later. Those future symbols can move the posterior, so a filter probability is the wrong label for a finished sequence.

Lecture 7 · 90 minutes

Ensembles

Bagging trains several classifiers on bootstrap draws and averages them. You draw a set of size n from a training set of size n, with replacement. The chance a given point is missed by one draw is about 1 over e, near 0.37, so those out-of-bag points can score that member. Averaging 10 trees cuts variance when their errors are not the same error. It does not cancel a bias every tree shares.

Boosting fits the next member to points the current ensemble still misses. When a member has weighted error 0.25, its coefficient is 0.5 times the natural log of 3, about 0.549. You then multiply a missed point's weight by the square root of 3, about 1.732, before renormalizing. A member with error 0.5 gets coefficient 0. You stop when held-out error rises, even if training error is still falling.

Two ways to combine classifiers.

Bagging

Bootstrap draws of size n

About 0.37 left out

Average the members

Boosting

Reweight missed points

Coefficient from the error

Stop on held-out error

One AdaBoost multiplier

1. A point starts with weight 1. The new member has weighted error 0.25 and misclassifies the point.

2. The coefficient is 0.5 times ln(3), about 0.549. Multiplying the weight by exp of that coefficient multiplies it by the square root of 3, about 1.732.

3. A point this member got right is multiplied by 1 over the square root of 3, about 0.577. After you

renormalize across the set, the missed point takes a larger share of the next fit.

Problem 1. Weighted error is 0.25. What exact multiplier hits a misclassified point's weight before renormalization?

Answer. The square root of 3, about 1.732. It equals exp of 0.5 times ln(3).

Problem 2. All 10 trees in a bagged ensemble make the same mistake on a point. What does the average predict, and which error remains?

Answer. The average repeats that mistake. Bagging only reduces variance that is not shared. The common bias stays.

Lecture 8 · 90 minutes

A pipeline you can defend

You can now turn a measurement into a feature vector, estimate a density with a histogram or a kernel, and pick a class by a Bayes rule or a linear score. You can cluster with k-means, read a variance fraction such as 0.8 off a principal component, filter a hidden Markov model and normalize it, and combine classifiers by bagging or by boosting. Each step has a number you can recompute.

You can also name the failure that makes the pipeline lie. A scale fit on the test split, a histogram bin of height 0 inside a logarithm, a k equal to the number of points, a filter used as a smoothed label, and an ensemble stopped on training error are five different mistakes. The work you can do is to run one dataset through two of these stages and report the held-out number you defined before you looked.

Two stages and a held-out number.

Fit the feature scale -> Estimate class densities -> Apply the Bayes rule -> Report held-out error

Scale, then a posterior

1. Training mean is 0 and training scale is 2. A raw value of 4 becomes (4 minus 0) divided by 2, so 2. You do not refit that scale on the test point.

2. At the scaled value 2, the class-conditional density is 0.3 for A and 0.1 for B. Priors are equal, 0.5 each.

3. The likelihood ratio is 0.3 divided by 0.1, so 3. With equal priors the posterior of A is 3 divided by 4, so 0.75. That 0.75 uses a representation fixed on training data.

Problem 1. Combine standardization and the Bayes rule. Raw value 4, training mean 0, scale 2, densities 0.3 and 0.1, equal priors. What is the posterior of A?

Answer. 0.75. The scaled value is 2. The likelihood ratio is 3, and 3 divided by 3 plus 1 is 0.75.

Problem 2. K-means training cost falls when you raise k from 2 to 20 on 20 points, and a classifier's training error falls to 0. Which drop tells you about new points?

Answer. Neither. K-means training cost falls as k grows, and training error can fall to 0 by memorizing. You need the held-out score you fixed before looking. Those two training drops are the clustering objective and the classifier fit from earlier lectures, not evidence on new points.